Trabalho 4 - Análise exploratória e classificação no dataset Iris

CI1026 - Visão Computacional — TA04

Bruno Wavrzenczak Edison Luiz Matias Junior (GRR20211790)
Efi Fuchs Santos da Silva (GRR20211775)

Repositório GitHub: https://github.com/BrunoWavrzenczak/Trabalho-4---Visao-Computacional-CI1026.git

Sumário

1 Introdução **2**

2 1. Carregamento e estatísticas descritivas **2**
2.1 1.1 Frequência por categoria . . . 3
2.2 1.2 Média, desvio padrão e moda geral . . . 3
2.3 1.3 Média, desvio padrão e moda por categoria . . . 4

3 2. Relação entre comprimento e largura da sépala **4**
3.0.1 Observações: . . . 5

4 3. Relação entre comprimento e largura da pétala **6**
4.0.1 Observações: . . . 6

5 4. Distribuição das categorias nas quatro dimensões **7**
5.0.1 Observações: . . . 8

6 5. Análise de componentes principais (PCA) **8**
6.0.1 Resultados do PCA: . . . 9

7 6. Divisão em treino e teste (80% / 20%) **10**

8 7. Classificação com K-Nearest Neighbors (K-NN) 10
8.1 7.1 Curva de acurácia vs. valor de k 12
8.2 7.2 Avaliação detalhada do modelo ($k = 7$) 12

9 Conclusões 14

1 Introdução

Este relatório apresenta uma análise exploratória dos dados e experimentos de classificação com o clássico dataset **Iris** (Fisher, 1936).

O conjunto possui 150 amostras divididas igualmente entre 3 espécies de flores (*Setosa*, *Versicolor* e *Virginica*), com 4 atributos contínuos mensurados em centímetros: - Comprimento da sépala (`sepal_length`) - Largura da sépala (`sepal_width`) - Comprimento da pétala (`petal_length`) - Largura da pétala (`petal_width`)

---

2 1. Carregamento e estatísticas descritivas

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Carregar base
iris = load_iris(as_frame=True)
df = iris.frame.rename(columns={
    'sepal length (cm)': 'sepal_length',
    'sepal width (cm)': 'sepal_width',
    'petal length (cm)': 'petal_length',
    'petal width (cm)': 'petal_width'
})
```

```
df['species'] = df['target'].map(dict(enumerate(iris.target_names)))
df = df.drop(columns=['target'])
features = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
```

2.1 1.1 Frequência por categoria

```
freq = df['species'].value_counts().to_frame(name='Frequência')
freq['Proporção (%)'] = (freq['Frequência'] / len(df)) * 100
freq
```

species	Frequência	Proporção (%)
setosa	50	33.333333
versicolor	50	33.333333
virginica	50	33.333333

A base é perfeitamente balanceada: exatamente 50 amostras (33,33%) para cada uma das 3 classes.

2.2 1.2 Média, desvio padrão e moda geral

```
def get_stats(data_subset, cols):
    mean_val = data_subset[cols].mean()
    std_val = data_subset[cols].std()
    mode_val = data_subset[cols].mode().iloc[0]
    return pd.DataFrame({'Média': mean_val, 'Desvio padrão': std_val, 'Moda': mode_val})

get_stats(df, features)
```

	Média	Desvio padrão	Moda
sepal_length	5.843333	0.828066	5.0
sepal_width	3.057333	0.435866	3.0
petal_length	3.758000	1.765298	1.4
petal_width	1.199333	0.762238	0.2

2.3 1.3 Média, desvio padrão e moda por categoria

```
for sp in df['species'].unique():
    print(f"=== Espécie: {sp.upper()} ===")
    display_df = get_stats(df[df['species'] == sp], features)
    print(display_df.round(3))
    print("\n")
```

```
=== Espécie: SETOSA ===
              Média  Desvio padrão  Moda
sepal_length  5.006          0.352   5.0
sepal_width   3.428          0.379   3.4
petal_length  1.462          0.174   1.4
petal_width   0.246          0.105   0.2


=== Espécie: VERSICOLOR ===
              Média  Desvio padrão  Moda
sepal_length  5.936          0.516   5.5
sepal_width   2.770          0.314   3.0
petal_length  4.260          0.470   4.5
petal_width   1.326          0.198   1.3

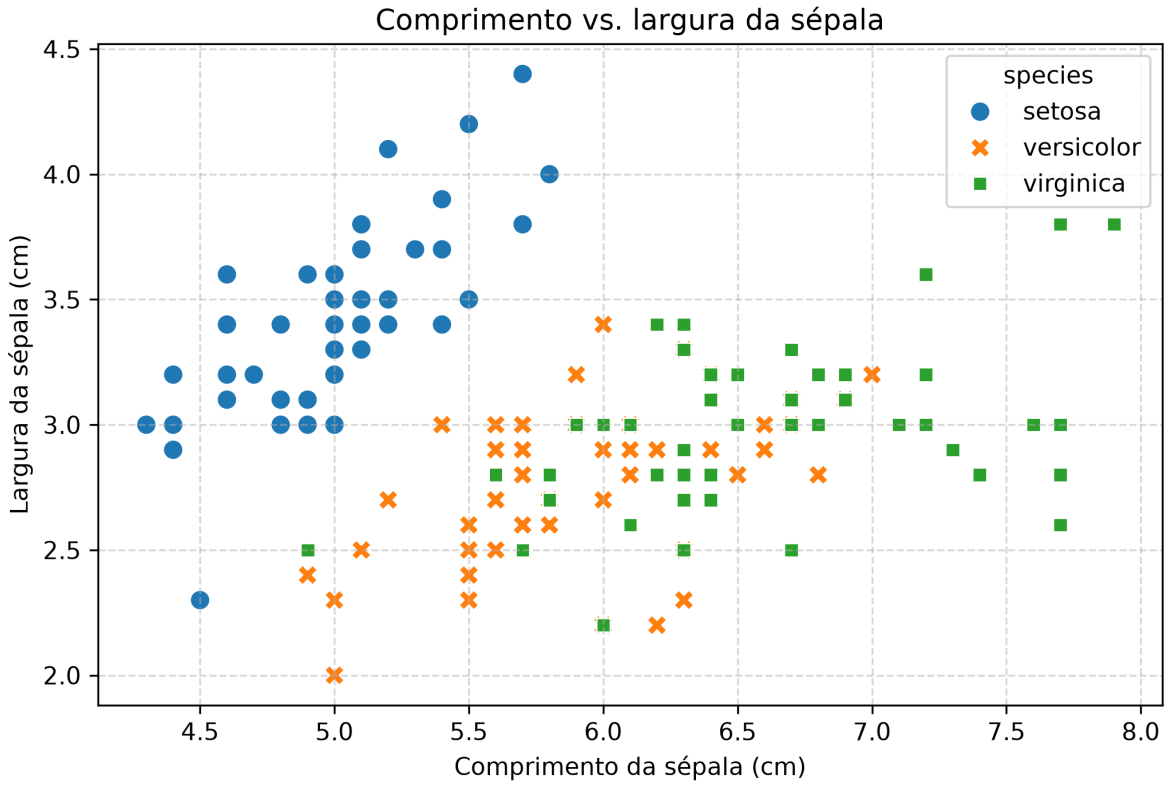

=== Espécie: VIRGINICA ===
              Média  Desvio padrão  Moda
sepal_length  6.588          0.636   6.3
sepal_width   2.974          0.322   3.0
petal_length  5.552          0.552   5.1
petal_width   2.026          0.275   1.8
```

---

3 2. Relação entre comprimento e largura da sépala

Analisamos a dispersão entre o comprimento e a largura da sépala para cada categoria:

```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='sepal_length', y='sepal_width', hue='species', style='species', s
plt.title("Comprimento vs. largura da sépala")
plt.xlabel("Comprimento da sépala (cm)")
plt.ylabel("Largura da sépala (cm)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



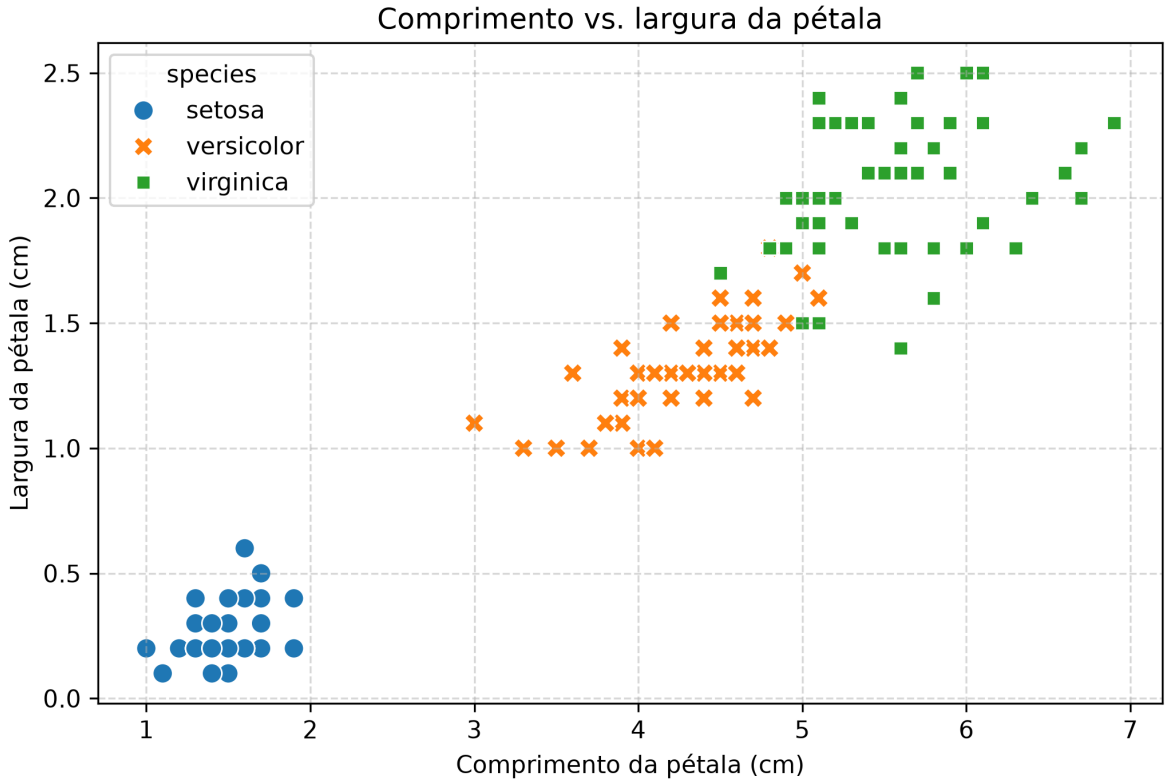
3.0.1 Observações:

- **Setosa**: Possui sépalas curtas, porém nitidamente mais largas que as demais espécies.
- **Versicolor e Virginica**: Há sobreposição considerável em ambas as dimensões de sépala, tornando a separação entre essas duas espécies difícil apenas pelas sépalas.

---

4 3. Relação entre comprimento e largura da pétala

```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='petal_length', y='petal_width', hue='species', style='species', s
plt.title("Comprimento vs. largura da pétala")
plt.xlabel("Comprimento da pétala (cm)")
plt.ylabel("Largura da pétala (cm)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



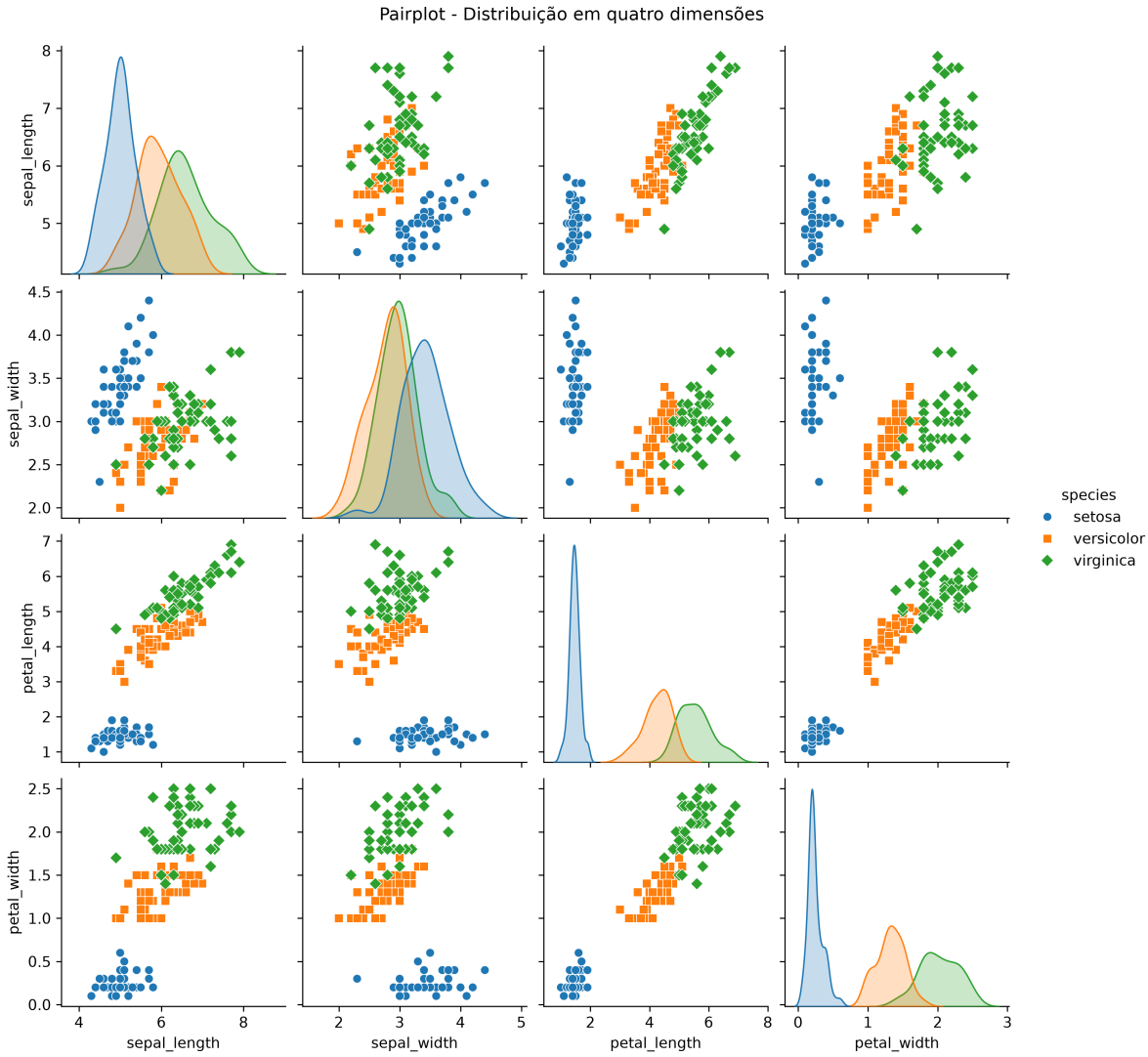
4.0.1 Observações:

- As dimensões das pétalas são **altamente discriminantes**.
- A espécie **Setosa** forma um cluster perfeitamente isolado no canto inferior esquerdo (pétalas pequenas).
- **Versicolor** e **Virginica** apresentam relação linear positiva forte e são quase linearmente separáveis, com apenas uma leve região de fronteira compartilhada.

5 4. Distribuição das categorias nas quatro dimensões

Para visualizar simultaneamente as relações bivariadas e as distribuições univariadas marginais de todas as 4 dimensões:

```
sns.pairplot(df, hue='species', markers=['o', 's', 'D'], corner=False)
plt.suptitle("Pairplot - Distribuição em quatro dimensões", y=1.02)
plt.show()
```



5.0.1 Observações:

- Os histogramas marginais (diagonal) mostram que `petal_length` e `petal_width` são as variáveis mais informativas para separação entre espécies.
- A espécie *Iris Setosa* é linearmente separável das outras duas em quase todas as projeções que envolvem as pétalas.

---

6 5. Análise de componentes principais (PCA)

O **PCA (Principal Component Analysis)** é uma técnica não-supervisionada de redução de dimensionalidade que projeta os dados em direções ortogonais de máxima variância (componentes principais).

```
# Padronização dos atributos (média 0, variância 1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[features])

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

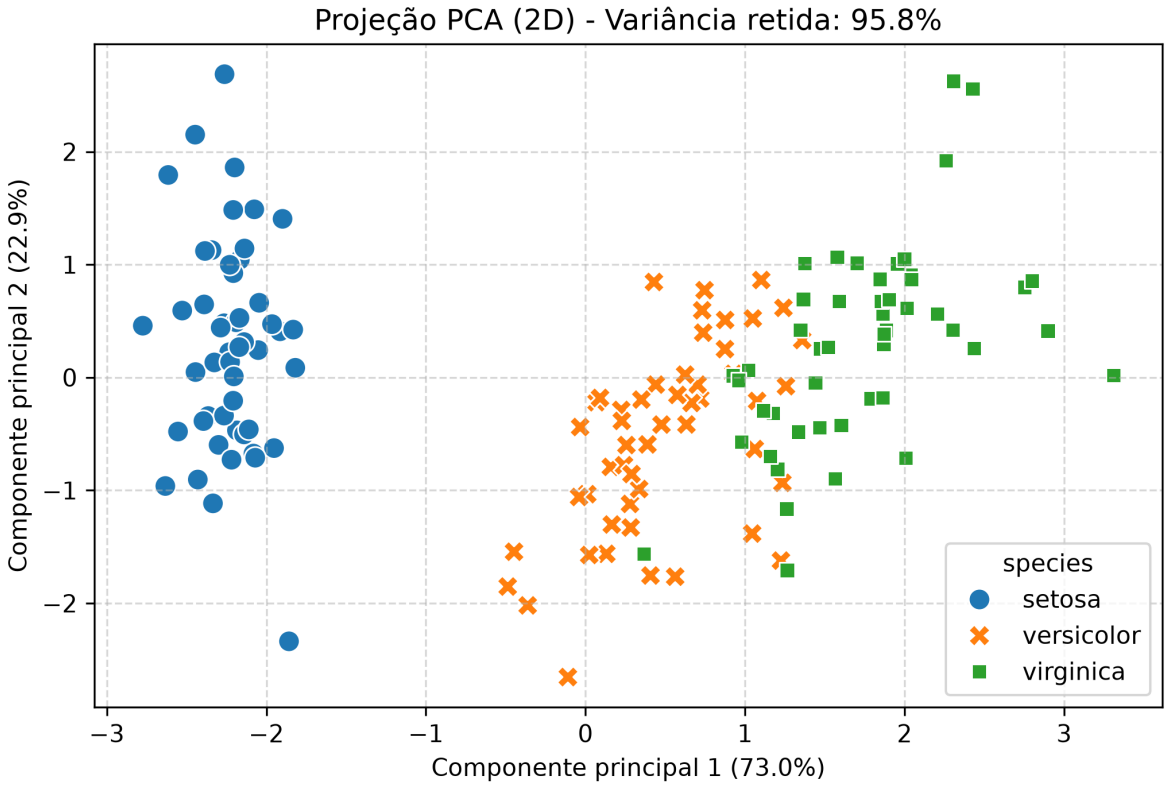
pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
pca_df['species'] = df['species']

var_pc1 = pca.explained_variance_ratio_[0] * 100
var_pc2 = pca.explained_variance_ratio_[1] * 100
total_var = var_pc1 + var_pc2

print(f"Variância explicada por PC1: {var_pc1:.2f}%")
print(f"Variância explicada por PC2: {var_pc2:.2f}%")
print(f"Variância total explicada (2 componentes): {total_var:.2f}%")
```

```
Variância explicada por PC1: 72.96%
Variância explicada por PC2: 22.85%
Variância total explicada (2 componentes): 95.81%
```

```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=pca_df, x='PC1', y='PC2', hue='species', style='species', s=80)
plt.title(f"Projeção PCA (2D) - Variância retida: {total_var:.1f}%")
plt.xlabel(f"Componente principal 1 ({var_pc1:.1f}%)")
plt.ylabel(f"Componente principal 2 ({var_pc2:.1f}%)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



6.0.1 Resultados do PCA:

- **Redução de 4D para 2D retendo 95,8% da informação/variância total**.
- No plano 2D, a espécie *Setosa* fica completamente afastada das outras duas.
- *Versicolor* e *Virginica* ficam claramente ordenadas ao longo do eixo PC1, permitindo excelente separação com pouquíssima sobreposição.

---

7 6. Divisão em treino e teste (80% / 20%)

```
X = df[features]
y = df['species']

# Divisão estratificada para manter o balanceamento das classes
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print(f"Total amostras de treino: {len(X_train)} (80%)")
print(f"Total amostras de teste:  {len(X_test)} (20%)")
print("\nDistribuição no teste:")
print(y_test.value_counts())
```

```
Total amostras de treino: 120 (80%)
Total amostras de teste:  30 (20%)

Distribuição no teste:
species
setosa        10
virginica     10
versicolor    10
Name: count, dtype: int64
```

---

8 7. Classificação com K-Nearest Neighbors (K-NN)

Avaliamos o classificador K-NN variando o número de vizinhos k de 1 a 50 (apenas valores ímpares: $k \in \{1, 3, 5, \ldots, 49\}$):

```
# Padronização ajustada no treino e aplicada ao teste
scaler_knn = StandardScaler()
X_train_scaled = scaler_knn.fit_transform(X_train)
X_test_scaled = scaler_knn.transform(X_test)

k_values = [k for k in range(1, 51, 2)]
results = []
```

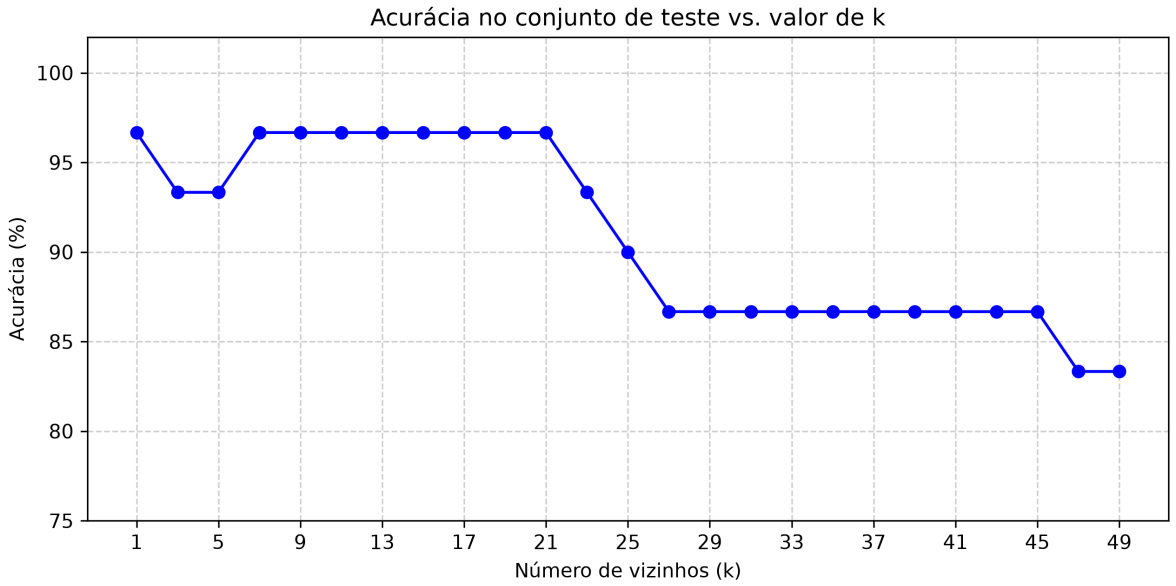
```
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    preds = knn.predict(X_test_scaled)
    acc = accuracy_score(y_test, preds)
    results.append({'k': k, 'Acurácia (%)': round(acc * 100, 2)})

df_results = pd.DataFrame(results)
df_results
```

	k	Acurácia (%)
0	1	96.67
1	3	93.33
2	5	93.33
3	7	96.67
4	9	96.67
5	11	96.67
6	13	96.67
7	15	96.67
8	17	96.67
9	19	96.67
10	21	96.67
11	23	93.33
12	25	90.00
13	27	86.67
14	29	86.67
15	31	86.67
16	33	86.67
17	35	86.67
18	37	86.67
19	39	86.67
20	41	86.67
21	43	86.67
22	45	86.67
23	47	83.33
24	49	83.33

8.1 7.1 Curva de acurácia vs. valor de k

```
plt.figure(figsize=(10, 4.5))
plt.plot(df_results['k'], df_results['Acurácia (%)'], marker='o', linestyle='-', color='b')
plt.title("Acurácia no conjunto de teste vs. valor de k")
plt.xlabel("Número de vizinhos (k)")
plt.ylabel("Acurácia (%)")
plt.xticks(k_values[::2])  # Exibir rótulos alternados para legibilidade
plt.ylim(75, 102)
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```



8.2 7.2 Avaliação detalhada do modelo ($k = 7$)

Como observado na curva de acurácia, para valores de k entre 7 e 21 a acurácia se mantém em patamar elevado e estável (96,67%). Avaliamos a matriz de confusão e métricas para $k = 7$:

```
selected_k = 7
best_knn = KNeighborsClassifier(n_neighbors=selected_k)
best_knn.fit(X_train_scaled, y_train)
y_pred = best_knn.predict(X_test_scaled)

print("Matriz de confusão:")
```

```
cm = confusion_matrix(y_test, y_pred, labels=iris.target_names)
cm_df = pd.DataFrame(cm, index=iris.target_names, columns=iris.target_names)
print(cm_df)

print("\nRelatório de classificação:")
print(classification_report(y_test, y_pred))
```

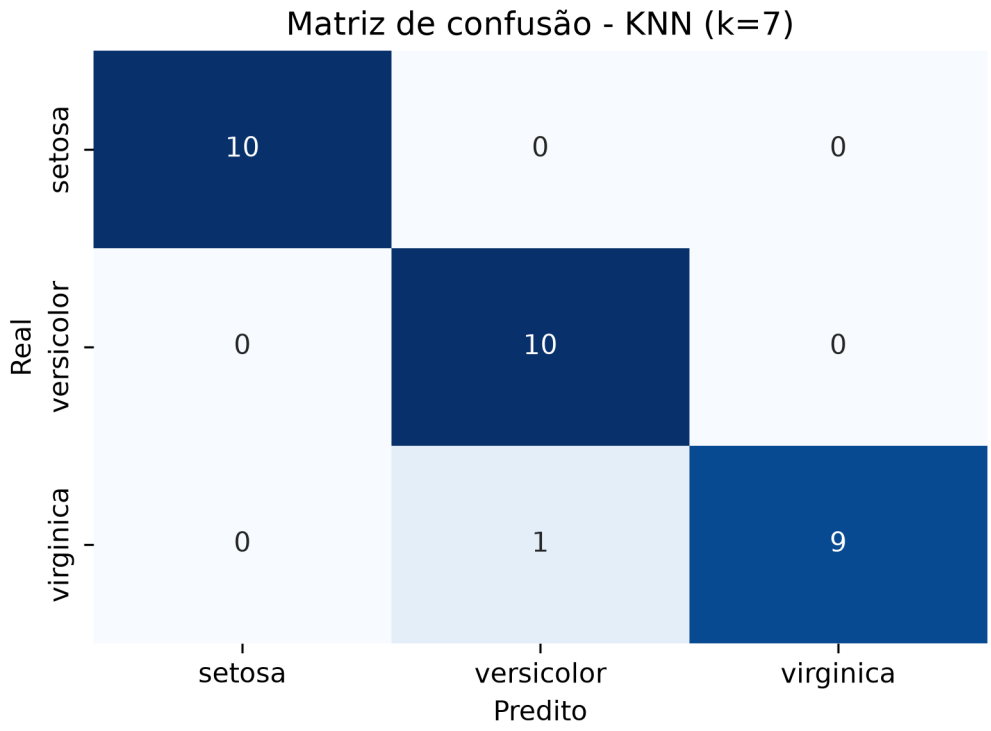
```
Matriz de confusão:
            setosa  versicolor  virginica
setosa          10           0          0
versicolor       0          10          0
virginica        0           1          9

Relatório de classificação:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.91      1.00      0.95        10
   virginica       1.00      0.90      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30
```

```
plt.figure(figsize=(6, 4))
sns.heatmap(cm_df, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title(f"Matriz de confusão - KNN (k={selected_k})")
plt.xlabel("Predito")
plt.ylabel("Real")
plt.show()
```



---

9 Conclusões

1. **Atributos mais relevantes**: As características da pétala (`petal_length` e `petal_width`) são as mais eficazes para discriminação entre as três classes.
2. **PCA**: Apenas 2 componentes principais conseguem capturar quase 96% de toda a variância contida nas 4 variáveis originais.
3. **Classificação K-NN**:
   - Para valores baixos a intermediários ($k \in [1, 21]$), o modelo apresenta acurácia alta e estável entre **93,3%** e **96,7%**.
   - À medida que k cresce acima de 23, a acurácia cai progressivamente até atingir **83,3%** em $k = 49$. Isso ocorre porque uma vizinhança excessivamente ampla passa a incluir pontos de classes distintas (especialmente entre *Versicolor* e *Virginica*), suavizando em excesso as fronteiras de decisão (subajuste / *underfitting*).